Explaining Genetic Programming-Evolved Routing Policies for Uncertain Capacitated Arc Routing Problems

Shaolin Wang, *Graduate Student Member, IEEE,* Yi Mei, *Senior Member, IEEE,* and Mengjie Zhang, *Fellow, IEEE*

***Abstract*—Genetic programming has been successfully used to evolve routing policies that can make real-time routing decisions for uncertain arc routing problems. Although the evolved routing policies are highly effective, they are typically very large and complex, and hard to be understood and trusted by real users. Existing studies have attempted to improve the interpretability by developing new genetic programming approaches to evolve both effective and interpretable (e.g., with smaller program size) routing policies. However, they still have limitations due to the trade-off between effectiveness and interpretability. To address this issue, we propose a new post-hoc explanation approach to explaining the effective but complex routing policies evolved by genetic programming. The new approach includes a local ranking explanation and a global explanation module. The local ranking explanation uses particle swarm optimisation to learn an interpretable linear model that accurately explains the local behaviour of the routing policy for each decision situation. Then, the global explanation module uses a clustering technique to summarise the local explanations into a global explanation. The experimental results and case studies on the benchmark datasets show that the proposed method can obtain accurate and understandable explanations of the routing policies evolved for uncertain arc routing problems. Our explanation approach is not restricted to uncertain arc routing, but has a great potential to be generalised to other optimisation and machine learning problems such as learning classifier systems and reinforcement learning.**

***Index Terms*—Capacitated Arc Routing, Genetic Programming, Hyper-Heuristic, Interpretability.**

I. Introduction

UNCERTAIN Capacitated Arc Routing Problems (UCARPs) [1] are the uncertain variant of the well-known CARPs [2], which is to serve a set of tasks (required edges) using a fleet of vehicles with the minimum cost. They have a wide range of real-world applications, such as winter gritting [3] and waste collection [4]. In contrast to CARPs, UCARPs assume that the parameters, such as task demand and travel cost, are stochastic. As a result, *route failures* can occur when the actual demand of a task is greater than expected, which leaves the vehicle with insufficient capacity to serve the task. The vehicle has to return to the depot to replenish its capacity and then come back to complete the failed service. This leads to a large recourse cost. How to handle route failures is the main challenge in UCARPs.

Most traditional optimisation methods (e.g., [5], [6]) designed for (static) CARPs are not effective enough to handle the route failures in UCARPs [7]. These methods generally require a solution to be optimised in advance. When a routing failure occurs, a recourse operator is used to repair the solution. However, a recourse operator is not powerful enough to fix the preplanned solution very well. Therefore, these methods are not well suited to UCARPs.

As a completely reactive approach, using routing policies [8] is a promising technique to tackle the uncertain and dynamic environment of UCARPs. When a vehicle becomes idle and there are candidate tasks that can be served by the vehicle, the routing policy is used to help the vehicle decide the next candidate task to serve in real time. In other words, the routing policy constructs the routes gradually, and each decision is made based on up-to-date information of the environment.

The effectiveness of routing policy heavily relies on instance characteristics such as graph topology and variable distributions [9]. Genetic Programming (GP) has been shown to be an effective Hyper-Heuristic (GPHH) approach to automatically designing routing policies for specific UCARPs instances through an offline training process [7].

Although the GP-evolved routing policies have shown effectiveness, they are often too complex to be understood by real users. With the use of Artificial Intelligence (AI), interpretability has become more and more important. Interpretability is the degree to which a human can understand the cause of a decision [10]. If people are not able to understand the principles behind the operation of AI models, it is difficult for these models to be trusted by users. As a result, the interpretability of AI models has become a challenge between AI models and their practical application. Therefore, it is important to improve the interpretability of GP routing policies for UCARPs.

The methods to improve the interpretability of AI models can be normally divided into two categories [11], i.e., intrinsic methods and post-hoc methods. Intrinsic methods are mainly used *during* the training process of the model. They design

Manuscript received XXX; revised XXX and XXX; accepted XXX. This work was supported in part by the Marsden Fund of New Zealand Government under Contract VUW1509 and Contract VUW1614 . (*Corresponding author: Shaolin Wang.*)

Shaolin Wang, Yi Mei, and Mengjie Zhang are with the Evolutionary Computation Research Group, School of Engineering and Computer Science, Victoria University of Wellington, Wellington 6140, New Zealand (e-mail: shaolin.wang@ecs.vuw.ac.nz; yi.mei@ecs.vuw.ac.nz; mengjie.zhang@ecs.vuw.ac.nz).

Colour versions of one or more of the figures in this article are available online at XXX.

Digital Object Identifier XXX

specific learning/search mechanisms to learn both effective and interpretable models [12]. The post-hoc methods, on the other hand, are mainly used *after* the model training process to "explain" the trained models. Post-hoc methods can be applied to not only "black-box" models, but also transparent models (the model itself is easy to interpret), since post-hoc methods are usually decoupled from the main model [12].

There have been a number of intrinsic methods [13], [14] proposed to improve the interpretability of GP-evolved routing policies. These methods generally improve interpretability by evolving smaller (simpler) routing policies. However, even with the effort to evolve simpler routing policies, the final routing policy may still be difficult to understand in some cases, as the GP-evolved routing policies need to maintain a certain level of complexity in order to be effective [15], [16]. Once we have evolved an effective GP-evolved routing policy for a UCARP instance, it becomes difficult to improve its interpretability without sacrificing its effectiveness. Contrary to the intrinsic methods, the post-hoc methods can potentially improve interpretability without sacrificing effectiveness, since they do not change the trained model.

Therefore, in this paper we focus on developing post-hoc methods to explain GP-evolved UCARP routing policies.

Under a UCARP decision situation, e.g., when a vehicle completes the current task and becomes idle, it uses a routing policy to prioritise/rank the candidate tasks (those that can be served by the vehicle without violating the capacity constraint) and selects the top-ranked task to serve next. Thus, the behaviour of a routing policy can be characterised from two aspects: (1) its task ranking and (2) the selected task. To explain the behaviour of a routing policy in a decision situation, we need to explain why the routing policy ranks the candidate tasks in a particular way, and why it selects a certain task over others in this decision situation.

In addition to explaining the above *local* behaviour of routing policy in each decision situation, it is also important to explain its *global* behaviour in all the different decision situations occurred during the entire service process. However, the global explanation is much more difficult to obtain than the local explanation for each single decision situation [17].

In this paper, we propose a new Local-Global Ranking Explanation (LGRE) method to tackle the above challenges. First, we define the local explanation as a ranking regression problem with a new fitness function to reflect the ranking and task selection behaviours. Then, we use Particle Swarm Optimisation (PSO) to evolve local explanations with the new fitness function, aiming to evolve linear models that can accurately match the local behaviour of the routing policy in the decision situation with the least number of attributes. Then, we collect all the local explanations obtained by PSO on all the decision situations occurred in each instance, and summarise them into a global explanation.

The overall goal of this paper is to develop new post-hoc methods (i.e., LGRE) to explain the local and global behaviours of GP-evolved rules for stochastic optimisation and machine learning problems, using UCARP as an example. We have the following specific research objectives:

1) To propose two measures, namely *consistency* and *correlation*, to characterise the local behaviour of routing policy in each decision situation.
2) To use PSO to evolve linear models for the local explanation, and define a new fitness function that incorporates the consistency, correlation and number of used attributes in the linear model.
3) To develop clustering mechanisms to summarise the local explanations of all the decision situations occurred in the UCARP instance into a global explanation, in the form of a production rule, for example, "If the decision situation shows pattern X, then the local explanation shows pattern Y".
4) To verify the accuracy of the proposed LGRE method, in terms of consistency and correlation in different decision situations.
5) To show the interpretability of the LGRE method by giving case studies of the local and global explanations.

It is important to note that although the new framework proposed in this paper is to generate local and global explanations of GP-evolved routing policies for UCARP, it is inherently general and has a great potential to be generalised to other problems, such as dynamic flexible job shop scheduling problem [18], dynamic cloud workflow scheduling problem [19], learning classifier systems [20] and reinforcement learning tasks [21].

II. Background

A. Uncertain Capacitated Arc Routing Problem

A UCARP instance is represented by a connected undirected graph $G(V, E)$, where V represents all the vertices in the graph and E represents all the edges in the graph. Each edge $e \in E$ has a deadheading cost $C_d(e)$, which refers to the cost to traverse the edge without serving it. A subset of N tasks $T \subseteq E$ ($|T| = N$) needs to be served. Each task $t \in T$ has a demand $D(t)$ and a serving cost $sc(t)$. Both $C_d(e)$ and $D(t)$ are unknown in advance, and they are modelled using a random distribution. The cost $\delta(t_1, t_2)$ from task t_1 to t_2 can be obtained by the shortest path finding algorithms (e.g., Dijkstra's algorithm) on the adjacency matrix of the deadheading cost during preprocessing. A fleet of vehicles with a capacity Q are allocated at the *depot* $v_0 \in V$ to serve all the tasks. The actual deadheading cost of an edge is observed after it is traversed. However, the realised demand of a task is unknown until a vehicle has arrived at the corresponding edge.

The main goal of solving this problem is to serve all the tasks by the vehicles with the least cost (serving cost plus deadheading cost of the shortest paths between tasks) under the following constraints:

- A vehicle's route must start and end at the depot;
- The total demand served by each route cannot exceed the vehicle's capacity.

In a UCARP instance *sample*, either the task demand $D(t)$ or deadheading cost $C_d(e)$ takes a realised value. However, the realised demand of a task is unknown until the vehicle arrived at the corresponding edge, and the realised deadheading cost

of an edge is unknown in advance and is revealed during the traversal over the edge. The actual value of them are sampled using a random distribution. They are defined as two types of *random variables* for UCARP in [1], [22]. These two types of random variables can cause four uncertain factors, i.e., stochastic task demand, stochastic edge deadheading cost, stochastic task presence and stochastic edge existence, where stochastic task demand and stochastic task presence is caused by random demand, and stochastic edge deadheading cost and stochastic edge existence are caused by random deadheading cost [1], [22].

The random variables can cause the violation of the constraints, which are called *route failures* and *edge failures*. A route failure occurs when the actual demand of a task being served by a vehicle is larger than expected and exceeds the vehicle's remaining capacity. Fig. 1 shows an example where the vehicle is serving the red edge with expected demand of 3 but actual demand of 5. The vehicle's remaining capacity of 4, thus is expected to be able to serve the task. However, the actual demand is larger than the remaining capacity, and a route failure occurs. In this case, the vehicle has to return to the depot to replenish its capacity in the middle of the service. An edge failure occurs when an edge in the route becomes inaccessible (its deadheading cost becomes infinity). The edge failure can be addressed by finding the shortest detour under the current situation using real-time path-finding algorithms (e.g., Dijkstra's algorithm [23]). The goal of solving UCARP is to minimise the expected total cost of the solution over all the possible values of the random variables after addressing the route and edge failures. A formal problem formulation can be found in [14].

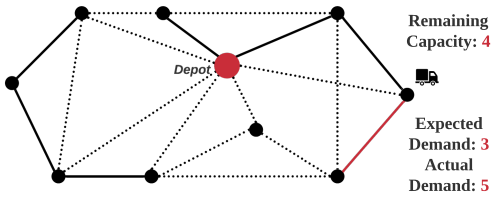


Fig. 1. An example of a route failure.

B. Genetic Programming Hyper Heuristic

GP Hyper Heuristic (GPHH) uses GP to evolve a population of heuristics. Instead of searching in the solution space, GPHH searches in the heuristic space. Each heuristic can construct a solution for a problem instance. GPHH has demonstrated its advantage in dealing with dynamic and stochastic problems and has been successfully applied not only to UCARP but also to many other problems, such as dynamic job shop scheduling problems [24], [25] and stochastic resource allocation problems [19].

When applying to UCARP, each individual of GPHH is a routing policy, which is essentially a priority function. Given a routing policy and UCARP instance, the solution is constructed as follows.

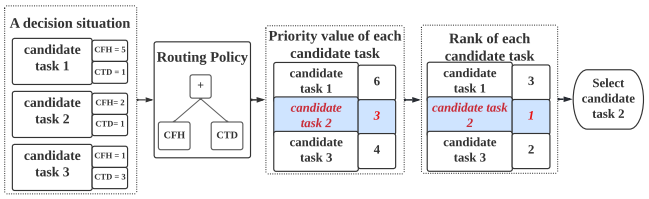


Fig. 2. An example of decision making process using a simple routing policy in a decision situation with three candidate tasks.

1) Initially, all the vehicles are at the depot, with idle time 0 and their routes are empty. All the tasks are unserved.
2) Find the *vehicle with the earliest idle time* (ties are broken by random selection), and its *candidate tasks*, i.e., the unserved tasks with expected demand less than its remaining capacity.
3) If no candidate task is found, send the vehicle back to the depot to replenish. Otherwise, use the routing policy to calculate the *priority* of each candidate task, and select the best-priority task to serve next. Set the idle time of the vehicle to the time it completes the service. Fig. 2 shows an example of using the CFH+CTD routing policy to select from 3 candidate tasks.
4) If all the tasks are served, send all the vehicles back to the depot and stop. Otherwise, go to step 2.

C. Related Work

1) Approaches to UCARP: UCARPs are uncertain variant of CARPs. Liu et al. [22] summarised that there are four uncertain factors in CARP, i.e., stochastic task demand, stochastic edge deadheading cost, stochastic task presence and stochastic edge existence, in their comprehensive review of Uncertain CARP. In this paper, UCARP refers to the model with all four uncertain factors.

In general, there are two main approaches to UCARP. The first type of approaches aims to find solutions that are expected to handle all the possible values of the random variables. The second type of approaches mainly uses GPHH to generate an effective routing policy, and uses the routing policy to construct the solution in real time.

The first type of approaches usually contains an optimisation algorithm and a recourse operator. The optimisation algorithm is utilised to search for a solution, and the recourse operator is utilised to repair the solution when failures occur. The performance of the proactive approaches can be highly impacted by the accuracy of the prediction of the stochastic environment as well as the effectiveness of the recourse operators. Fleury et al. [26] developed a novel Memetic Algorithm (MA) for CARPs with stochastic task demand (SCARP). This MA can handle the stochastic task demand in SCARP by utilising an accurate approximation of the expected total cost. This method utilises posterior techniques to handle uncertainties. In case of a violation of vehicle capacity before serving a task, the vehicle must return to the depot via the shortest path to refill and go back to serve it, then continue its planned trip. Wang et al. [27] also developed a MA for

UCARP. An integrated fitness function is proposed to direct the search process, and a large step-size local search operator is developed to prevent the search from being trapped in local optima. This approach utilises a resampling strategy to handle uncertainties. The solutions are evaluated on an identical set of scenarios for a fair comparison. Babaee et al. [28] proposed a hybrid meta-heuristic approach integrated with a mix integer programming model. This approach uses robust optimisation to handle uncertainties and assumes known deviation of demands and solved it approximately using a hybrid simulated annealing algorithm to optimise the solution constructed by a constructive heuristic. Wang et al. [29] developed an Estimation of Distribution Algorithm with Stochastic Local Search (EDASLS) for UCARP. The EDASLS is developed based on the edge histogram based sampling algorithm [30]. A novel stochastic local search is proposed and integrated to optimise solution that performs well over a set of UCARP instances. EDASLS also resampling strategy to handle uncertainties. It resamples UCARP instances for evaluating each solution and used its worst-case cost on the sampled instances for measure the solutions. Recently, Tong et al. [31] proposed a generalised meta-heuristic framework for UCARP. This framework uses either a simple restart strategy or a sequence transfer strategy to handle uncertainties. Both of them are reoptimisation strategy. The restart strategy starts from scratch without using any information. In contrast, the sequence transfer strategy generates a potentially good solution based on the previous optimisation experience by transferring the sequence of remaining unserved tasks. In addition, Tong et al. [32] proposed a hybrid local search framework which can handle small dynamic changes in UCARP. This approach reschedules the service plan in a short time when failure occurs. An archive is utilised to cover more search areas than single local search algorithms. The local search mechanisms is utilised to speed up the re-optimisation process of meta-heuristic.

The second type of approaches usually utilises GP to evolve routing policies. The evolved routing policy then can be applied to unseen or future scenarios to make routing decisions in real time. Weise et al. [33] firstly proposed a GPHH to a single-vehicle UCARP. The evolved routing policy outperforms manually designed routing policies. Then, Liu et al. [7] added a novel pool filer to GPHH to further improve the effectiveness of GPHH. The pool filer can filter out irrelevant candidate tasks during the decision-making process. To better reflect the reality, Mei et al. [9] extended GPHH from single-vehicle to multi-vehicle version. After that, MacLachlan et al. [34] proposed a GPHH with vehicle collaboration which allows multiple vehicle collaboration. Liu et al. [35] proposed a new GPHH paradigm, which utilises Cooperative Coevolution GP to evolve a task sequence and a recourse policy simultaneously. Recently, Ardeh et al. [36] concerned about the training speed of GPHH. Thus, they applied transfer learning techniques to GPHH. Besides, Wang et al. [14] take the interpretability of the GP-evolved routing policies into account. They proposed a new simplification method to evolved simpler routing policies.

2) Interpretability in GP: Genetic Programming (GP) is considered an interpretable approach since its flexible tree-based representation [37]. Therefore, it is often utilised to explain other "black-box" models, such as Deep Neural Networks (DNNs) [38], [39], Support Vector Machines (SVMs) [39], Random Forest (RF) [39] and Reinforcement Learning (RL) [40]. Evans et al. [38] proposed a global model extraction method which uses multi-objective genetic programming to construct accurate, simplistic and model-agnostic representations of complex black-box estimators. The result showed that the newly proposed method can offer drastically simpler models, with statistically equivalent test accuracy over DNNs and RF. Rather than reproducing a global model, Ferreira et al. [39] utilised GP to evolve a local explanation model. The Genetic Programming Explainer (GPX) was proposed to the problem of explaining decisions computed by AI systems. GPX generates a noise set located in the neighbourhood of the point of interest and fits a local explanation model for the analysed sample. The result showed that GPX can reflect the local behaviour of the complex models, such as DNNs, SVMs and RF.

In addition to helping to explain "black-box" models, GP can be also directly utilised to evolve interpretable programs for classification, regression and clustering problems. Berlanga et al. [41] utilised GP to evolve compact and accurate GP trees for high-dimensional classification problems. The evolved GP classification system showed a good performance in terms of accuracy and interpretability. Hein et al. [40] utilised GP to discover interpretable policies for RL. The authors proposed a Genetic Programming for Reinforcement Learning (GPRL) approach based on model-based batch reinforcement learning and genetic programming, which autonomously learns policy equations from pre-existing default state–action trajectory samples. GPRL can produce well-performing interpretable reinforcement learning policies from the pre-existing default state–action trajectory data. Hu et al. [42] designed a linear GP (LGP) algorithm and feature importance evaluation methods for the bioinformatics application problem of predicting disease risk using metabolite abundance levels in blood samples. LGP can evolve highly accurate classification models and the feature selection method based on feature importance evaluation can help further improve the prediction accuracy and discover the causality between the input and output. Zhao [43] utilised multi-objective GP to evolve a Pareto Front of interpretable decision tree-style rules for classification problem. Although GP can be utilised to evolve interpretable models, one issue is that the models become difficult to interpret when they become too complex. Wang et al. [13], [44], [45] intended to handle this issue by directly reducing the size of the evolved GP programs. In addition, the number of Splitting points was considered by Evans et al. [38]. Keijzer and Babovic [46] and Mei et al. [47] handled this issue by applying dimensionally aware GP which considers the physical dimensions of the problem features, and favours the combinations between the features with the same physical dimension. Alba et al. [48] and Hein et al. [49] utilised strongly-typed GP to constrain the combinations of different feature types to evolve interpretable GP programs. Hooper et al. [50] utilised the expression simplification to simplify the GP programs. Their simplification method employed over 200

simplification rules to simplify an expression. To speed up the simplification process, Zhang and Wong et al. [51], [52] proposed the use of algebraic GP programs simplification with hashing techniques and applied it to symbolic regression and classification problems. In addition to the algebraic approach, Kinzett [53] and Song et al. [54] proposed the numerical simplification methods. Wang et. al. [14] proposed a niching-based simplification method. A comprehensive survey in GP interpretability can be found in [55].

3) Interpretability in GPHH for UCARP: There are two categories of approaches to interpretability based on the techniques and the model properties [17]. The first one is mainly about the intrinsic which aims to the complexity of the model directly, and the other one consists of post-hoc explanations. Usually, these two categories can be distinguished by identifying whether interpretability is achieved through constraints imposed on the complexity of the ML model (intrinsic) or by applying methods that analyse the model after training (post-hoc) [11].

Intrinsic approaches mainly focus on the transparency of the model. Transparency refers to the degree that a model can be understood by users directly [12]. If an entire model can be understood and simulated by a person at once, then the model can be called transparent. In this case, an interpretable model should be a simple model. Thus, model complexity takes a dominant place in transparency [56]. For example, a rule-based system that contains a large number of simple rules is not transparent, however, a single perceptron neural network is transparent. This claim is consistent with the argument that simple linear models (having few coefficients) are more interpretable than complex linear models (having many coefficients) learned on the same inputs [57]. Intrinsic interpretability is typically achieved by imposing constraints on the model, such as monotonicity, causality, sparsity, or physical constraints that come from the domain knowledge [58].

Branke et. al. [15] investigated the use of a simple linear model as an alternative to the GP-evolved rules, however, the linear model gave significantly worse results than the GP-evolved model. Jia et. al. [16] also explored different representations of routing policies, and eventually found that a routing policy represented as a GP tree was the best choice. To evolve more interpretable routing policies for UCARP, Wang et al. [13], [14], [44] take the intrinsic interpretability of the GP-evolved routing policies into consideration by evolving simpler routing policies. Wang et al. [13], [44] firstly add size limitation for the GP tree to evolve smaller routing policies. In addition, Wang et al. [14] proposed a novel online simplification approach to simplify the routing policies to reduce the complexity of the evolved routing policies. Multi-objective methods [59], [60] were also proposed to evolve simpler routing policies. Both size and effectiveness of routing policies are utilised as two potentially conflicting objectives. A Pareto front of routing policies with different degrees of interpretability is evolved so that users can choose based on their preferences. Even though users can understand relatively simple routing policies, in some cases GP needs to evolve very complex routing policies to ensure effectiveness. It is necessary to find ways to explain these complex routing policies.

Post-hoc (post-model) interpretability refers to applying explanation methods to the learned model as some models are not readily interpretable by design. It can extract information from the learned model. Note that, post-hoc interpretations often do not elucidate precisely how a model works, they may nonetheless confer useful information for practitioners and end users of machine learning [17]. It can also answer the question what else the model can tell us. Post-hoc methods can be applied to not only "black-box" models, but also transparent models (the model itself is easy to interpret), since post-hoc methods are usually decoupled from the main model [12]. Some common approaches to post-hoc interpretations include text explanations, visualisations of learned representations or models, local explanations and example-based explanations [11], [17], [61].

To our best knowledge, there is only one existing post-hoc interpretability work for GPHH for UCARP. Wang et. al. [62] proposed a LASSO-based local explanation method for GP-evolved routing policies for UCARP. Local explanations handle the interpretability by explaining the learned models locally. Because it is quite difficult to describe the full mapping learned by a model, but it is much easier to give explanations to less complex solution sub-spaces that are relevant for the whole model [17]. The proposed method trains a LASSO regression model as a local explanation for a GP-evolved routing policy in a decision situation. It allows for effective local explanations. However, it still has limitations. Firstly, the performance of the proposed method has only been evaluated on a few instances. Secondly, its accuracy on matching the task ranking and selected tasks is not satisfactory enough.

In summary, we can see that previous research has focused on intrinsic methods, while post-hoc has yet to be investigated.

III. The Proposed Method

This section describes the framework of Local-Global Ranking Explanation (LGRE), a post-hoc interpretability method to help explain the decision made by routing policies. LGRE contains two layers. The first layer is a PSO-based searching algorithm to find the approximation of the original routing policy in a local decision situation. This PSO-based searching algorithm will be discussed in Section III-A. The second layer uses k-means clustering to cluster the large number of local explanations obtained in the first layer, and extract patterns from clusters to generate a global explanation, which will be discussed in Section III-B. There are two main differences between this work and [62]. First, the work in [62] modelled the local ranking explanation task as a normal regression task (linear model directly predicting the ranks). However, in this paper, the model is changed from a normal regression task to a learning-to-rank task and used PSO to learn the model. Second, the work in [62] is only a local explanation method. In contrast, this work extended it to both local and global explanation methods. The approach has also moved from a single method of providing the local explanation to a two-layer structured explanation framework. The first layer of the

framework generates a series of local ranking explanations by any learning-to-rank method such as PSO in this paper, and the second layer of the framework employs the local explanations obtained from the first layer to generate the global explanation.

A. Local Ranking Explanation

The process of the proposed local ranking explanation of a *routing policy* $\varphi(\mathbf{x})$ under a *decision situation* $\mathbf{X}$ is shown in Algorithm 1, and is illustrated in Fig. 3. Specifically, a routing policy $\varphi(\mathbf{x})$ is a priority function of the attributes $\mathbf{x}$, e.g., the serving cost the candidate task (SC) and demand of the candidate task (DEM). A decision situation $\mathbf{X} = [x_{ij}]_{M\times N}$ is a matrix with M candidate tasks and N attributes, where x_{ij} stands for the attribute j value of candidate task i in this decision situation. In the decision situation, a vehicle aims to select the next task to serve from the M candidate tasks. To explain the local behaviour of the routing policy in this decision situation, the local ranking explanation learns a *linear model* with as few features as possible because linear models are considered easy to construct, widely applied and "highly interpretable" [11], [63].

To explain the local behaviour accurately, the linear model is required to (1) select the same task as the routing policy, and (2) have the same task ranking as the routing policy. To this end, the local ranking explanation consists of *decision-specific attribute extraction*, *priority and rank calculation*, *linear model learning*, and *feature selection*, which will be described next.

1) Decision-specific Attribute Extraction: First, the *decision-specific attributes* $\mathbf{x}^{\langle local\rangle} \subseteq \mathbf{x}$ among the attributes appear in the GP tree to be explained. A decision-specific attribute is an attribute that is specific to the decision (i.e., candidate task in this case), such as the demand and location of the candidate task. On the contrary, a *decision-independent attribute* represents the global environment that is independent of the decision (candidate task), such as the remaining capacity and location of the vehicle. The corresponding decision-specific attribute columns in $\mathbf{X}$ are also extracted, and the resultant matrix is denoted as $\mathbf{X}^{\langle local\rangle}$. We have normalised the data and each decision-specific attribute has been normalised to 0 to 1 so that we can analyse the coefficients of the linear models directly.

2) Priority and Rank Calculation: Second, for each candidate task i ($i = 1, \ldots, M$), we calculate its priority value by $\varphi(\mathbf{x}_i)$, where $\mathbf{x}_i := \mathbf{X}_{i,:}$ is the feature vector of the task, i.e., row i of $\mathbf{X}$. Otherwise, use the routing policy to calculate the priority of each candidate task, and select the best-priority task to serve next. Ties are broken by selecting the task with the smallest ID, which is the simplest and most naive deterministic tie breaker. We do not consider random tie breaking that is less interpretable in practice, nor heuristic tie breaking, which may reduce the effect of GP-evolved policies (e.g., GP will learn policies to generate many ties and use the heuristic tie breaker to make decisions). Note that we use this tie breaker as an example (since the routing policies were trained under this tie breaker). However, our approach can theoretically be applied to any policy with any tie breaker, as long as we know how to break ties. We can simply change the task removal process by removing the tasks except for the one selected by the tie breaker.After removing the tasks with the same priority, all the remaining tasks have distinct priority values. Then, we calculate the task ranks $\boldsymbol{\gamma}$ by the $\texttt{sortedIdx}(\cdot)$ method. For example, the task with the minimal priority value has rank 1, and that with the second minimal priority value has rank 2.

Algorithm 1: Local Ranking Explanation

Input: A routing policy to be explained $\varphi(\mathbf{x})$, a decision situation $\mathbf{X} = [x_{ij}]_{M\times N}$
Output: A linear model to explain $\varphi(\mathbf{x})$ on $\mathbf{X}$

```
Identify the decision-specific attributes x^⟨local⟩ ⊆ x;
Let the decision-specific attribute columns in X be X^⟨local⟩;
// Priority and rank calculation
for i = 1 → M do
    Calculate priority φ(x_i) for task x_i := X_{i,:};
Let φ(X) := [φ(x_1), ..., φ(x_M)];
// Remove tie
for i_1 = 1 → M − 1 do
    for i_2 = i_1 + 1 → M do
        if φ(x_{i_1}) = φ(x_{i_2}) then
            Remove x_{i_2} from X;
            Remove φ(x_{i_2}) from φ(X);
γ = sortedIdx(φ(X));
// Linear model learning
β = PSOLE(X^⟨local⟩, γ);
// Feature selection
β = LEFS(β, X^⟨local⟩);
return the linear model x^⟨local⟩ · β̂;
```

3) Linear Model Learning: Third, we learn a linear model $\mathbf{x}^{\langle local\rangle} \cdot \boldsymbol{\beta}$ to fit the task ranks $\boldsymbol{\gamma}$. Different from traditional linear regression, which directly compares between the predicted values and the target values, we compare between the *ranks of the tasks* based on the predicted values given by the linear model and the target ranks. To this end, there are three steps to fit the task ranks:

1) Calculate the predicted task priority values $\mathbf{X}^{\langle local\rangle} \cdot \boldsymbol{\beta}$;
2) Convert the task priority values to task ranks by $\boldsymbol{\gamma}_{\boldsymbol{\beta}} = \texttt{sortedIdx}(\mathbf{X}^{\langle local\rangle} \cdot \boldsymbol{\beta})$;
3) Compare between $\boldsymbol{\gamma}_{\boldsymbol{\beta}}$ and the target task ranks $\boldsymbol{\gamma}$. Ideally, they should be the identical.

Due to the conversion from the raw priority value to the rank, the normal linear regression methods such as least squares and Lasso are no longer applicable. Therefore, we employ Particle Swarm Optimisation (PSO) [64], which is a promising gradient-free continuous optimisation method, to find the best $\boldsymbol{\beta}$ values.

The PSO-based Local Explanation, $\texttt{PSOLE}(\cdot)$, is described in Algorithm 2. It is a standard PSO process that evolves a swarm of particles $\mathbf{B} = [\beta_{ij}]_{N\times P}$, where P is the number of particles, and N is the number of attributes/variables in the linear model. The jth particle is represented by the jth column

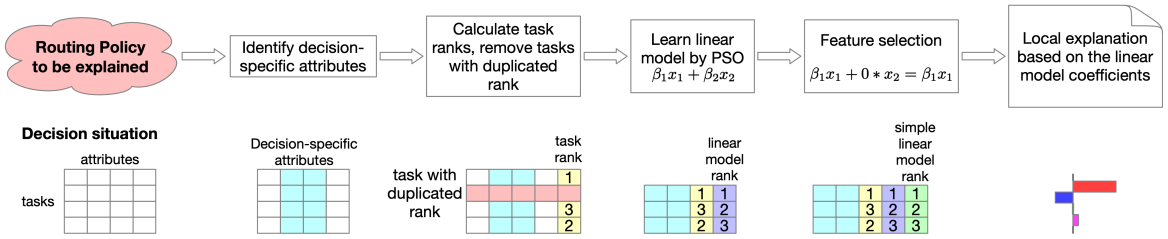


Fig. 3. The process of the local ranking explanation of a routing policy on a decision situation.

Algorithm 2: $\texttt{PSOLE}(\mathbf{X}, \gamma)$

Input: A decision situation $\mathbf{X} = [x_{ij}]_{M\times N}$, a rank vector γ
Output: The coefficients $\boldsymbol{\beta}$ of linear model $\mathbf{x}\cdot\boldsymbol{\beta}$

// Initialisation
Randomly initialise a population of weights $\mathbf{B} = [\beta_{ij}]_{N\times P}$ and velocities $\mathbf{V} = [v_{ij}]_{N\times P}$;
while *true* **do**
  Calculate output matrix $\mathbf{Y} = [y_{ij}]_{M\times P} = \mathbf{X}\cdot\mathbf{B}$;
  // Fitness evaluation
  for $j = 1 \to P$ **do**
    Get the particle output $\mathbf{y}_j = [y_{1j}, \ldots, y_{Mj}]$;
    $\gamma_j = \texttt{sortedIdx}(\mathbf{y}_j)$;
    Calculate $\varsigma(\gamma_j, \gamma)$ by Eq. (1);
    Calculate $\texttt{corr}(\gamma_j, \gamma)$ by Eq. (2);
    Calculate the fitness of $\boldsymbol{\beta}_j := \mathbf{B}_{:,j}$ by Eq. (3);
    Update pbest $\boldsymbol{\beta}_j^{\langle pb\rangle}$ and gbest $\boldsymbol{\beta}^{\langle gb\rangle}$;
  if *stop* **then return** $\boldsymbol{\beta}^{\langle gb\rangle}$;
  // Particle update
  for $j = 1 \to P$ **do**
    for $i = 1 \to N$ **do**
      $v_{ij} = wv_{ij} + r_1c_1\beta_{ij}^{\langle pb\rangle} + r_2c_2\beta_i^{\langle gb\rangle}$;
      $\beta_{ij} = \beta_{ij} + v_{ij}$;
  return $\boldsymbol{\beta}$;

of $\mathbf{B}$, i.e., $\boldsymbol{\beta}_j = \mathbf{B}_{:,j}$.

For evaluating a linear model $\mathbf{x}^{\langle local\rangle}\cdot\boldsymbol{\beta}$, we consider the following three aspects that affect the quality of explanation:

- *Consistency*: whether the linear model selects the same task (with rank 1) as the routing policy $\varphi(\cdot)$. A higher consistency indicates that the linear model makes more consistent decisions with the routing policy. The consistency between two rank vectors γ and γ' is defined as

$$\varsigma(\gamma, \gamma') = \begin{cases} 1, & \text{if } \texttt{idx}(\gamma, 1) = \texttt{idx}(\gamma', 1), \\ 0, & \text{otherwise}, \end{cases} \tag{1}$$

  where γ refers to the rank vector of routing policy and γ' refers to the rank vector of linear model, $\texttt{idx}(\gamma, 1)$ indicates the index of 1 in γ.
- *Correlation*: how close is the rank vector of the linear model to that obtained by the routing policy. A higher correlation indicates that the linear model behaves more closely related to the routing policy for all the candidate tasks in the decision situation. The correlation between two rank vectors γ and γ' is defined as follows:

$$\texttt{corr}(\gamma, \gamma') = \frac{\sum_{i=1}^{N}(\gamma_i - \bar{\gamma})(\gamma'_i - \bar{\gamma'})}{\sqrt{\sum_{i=1}^{N}(\gamma_i - \bar{\gamma})^2}\sqrt{\sum_{i=1}^{N}(\gamma'_i - \bar{\gamma'})^2}}. \tag{2}$$

  where $\bar{\gamma}$ and $\bar{\gamma'}$ refer to the average rank of the two rank vectors, respectively.
- *Model complexity*: How complex the model is. A less complex linear model tends to be easier to interpret. Here, the model complexity is measured by the L1 regularisation term $\sum_{i=1}^{N}|\beta_i|$. In case two models predict the same ranks (e.g., f(x) and 2*f(x)), the one with less complexity is considered better.

To take the above aspects into account, the fitness function of PSO is defined as follows:

$$\texttt{fit}(\boldsymbol{\beta}) = 1 - \frac{\varsigma(\gamma_{\boldsymbol{\beta}}, \gamma) + \texttt{corr}(\gamma_{\boldsymbol{\beta}}, \gamma)}{2} + \sum_{i=1}^{N}|\beta_i|. \tag{3}$$

The goal of PSO is to find the best linear model coefficients $\boldsymbol{\beta}^*$ that minimises the above fitness function. Note that PSO can be replaced by any other learning-to-rank methods [65] that is suitable to optimise the loss function as Eq. (3).

Remark. *The linear model only considers the decision-specific attributes. In other words, it does not consider the decision-independent attributes, and has no constant term. First, the task ranks are unchanged by adding any constant. Therefore, we can ignore the constant term in the linear model without changing the ranking. Second, in a decision situation, all the candidate tasks have the same decision-independent attribute values, thus the decision-independent attributes are constants for the task ranking and can be ignored.*

4) Feature Selection: Due to the sensitivity of the performance on the regularisation term $\sum_{i=1}^{N}|\beta_i|$, the $\boldsymbol{\beta}$ value obtained by PSO might still have some small non-zero β coefficients. Intuitively, simply changing a tiny β value to zero can greatly improve the interpretability of the linear model, as we can remove the corresponding feature. However, it is important to ensure that such feature removal does not degrade the accuracy of the model. To address this issue, we develop a local explanation feature selection $\texttt{LEFS}(\cdot)$ method to remove features without decreasing the model accuracy in terms of consistency and correlation.

The LEFS method is described in Algorithm 3. It repetitively identifies a feature so that after removing the feature, the

Algorithm 3: LEFS$(\boldsymbol{\beta}, \mathbf{X}, \boldsymbol{\gamma}, \varepsilon)$

Input: A weight vector $\boldsymbol{\beta} = [\beta_i]_{N\times 1}$, a decision situation $\mathbf{X} = [x_{ij}]_{M\times N}$, the target rank vector $\boldsymbol{\gamma}$, a correlation threshold ε

Output: An updated weight vector $\hat{\boldsymbol{\beta}}$

Calculate output $\mathbf{y}_{\boldsymbol{\beta}} = [y_i]_{M\times 1} = \mathbf{X}\cdot\boldsymbol{\beta}$;
The ranks $\boldsymbol{\gamma}_{\boldsymbol{\beta}} = \texttt{sortedIdx}(\mathbf{y})$;
Calculate the correlation $\texttt{corr}(\boldsymbol{\gamma}_{\boldsymbol{\beta}}, \boldsymbol{\gamma})$;
while *true* **do**
  Initialise $i^* = 0$, $\delta^* = \varepsilon$;
  for $i = 1 \to N$ **do**
    if $\beta_i = 0$ **then continue**;
    `// Try to remove feature` i
    Set $\boldsymbol{\beta}' = \boldsymbol{\beta}$, and change $\beta'_i = 0$;
    Calculate new output $\mathbf{y}_{\boldsymbol{\beta}'} = \mathbf{X}\cdot\boldsymbol{\beta}'$;
    $\boldsymbol{\gamma}_{\boldsymbol{\beta}'} = \texttt{sortedIdx}(\mathbf{y}_{\boldsymbol{\beta}'})$;
    if $\texttt{idx}(\boldsymbol{\gamma}_{\boldsymbol{\beta}}, 1) = \texttt{idx}(\boldsymbol{\gamma}_{\boldsymbol{\beta}'}, 1)$ **then**
      Calculate correlation loss $\delta = \texttt{corr}(\boldsymbol{\gamma}_{\boldsymbol{\beta}}, \boldsymbol{\gamma}) - \texttt{corr}(\boldsymbol{\gamma}_{\boldsymbol{\beta}'}, \boldsymbol{\gamma})$;
      if $\delta < \delta^*$ **then**
        Update $i^* = i$, $\delta^* = \delta$;
    if $i^* = 0$ **then break**;
    Set $\beta_i = 0$; `// Remove feature` i
Normalise the weights $\hat{\boldsymbol{\beta}} = \frac{\boldsymbol{\beta}}{\max(\boldsymbol{\beta})}$;
return $\hat{\boldsymbol{\beta}}$;

selected task (i.e., $\texttt{idx}(\boldsymbol{\gamma}_{\boldsymbol{\beta}}, 1)$) is unchanged, and the correlation obtained by the new model is not much worse (within a predefined threshold ε). If multiple features meet the condition, then the one with the least correlation decrease is removed (i.e., the corresponding β is set to 0). The feature selection process continues until no feature can be removed without deteriorating the model accuracy. Finally, the remaining weights are normalised, i.e., they are divided by the maximal weight, to make it easy to do clustering later. Note that the weight normalisation does not change the model accuracy, since the task ranks are not affected.

5) An Example Illustration: Fig. 4 shows an example to explain the routing policy $x_1+x_2+x_3*x_4$, where the meaning of the four attributes are as follows:

- x_1 (CFH): the cost from the vehicle's current location to the candidate task
- x_2 (DEM): the demand of the candidate task
- x_3 (RQ): the remaining capacity of the vehicle
- x_4 (PRT): the percentage of remaining (unserved) tasks

It can be seen that x_1 (CFH) and x_2 (DEM) are decision-specific attributes, while RQ and PRT are decision-independent attributes. After calculating the priority, we found that tasks 2 and 4 have the same priority value, thus task 4 is removed as it has a larger index than task 2. The resultant target rank vector is $[2, 3, 1]$ for the linear model learning. After applying the PSOLE method, we might obtain a linear model $x_1+0.01*x_2$, with perfect consistency and correlation. Finally, after the feature selection, we can further reduce β_2 to zero, and simplify the model to x_1 as there is no impact on the correlation and consistency when β_2 is removed. In other words, we explain that the routing policy selects the closest task to the current vehicle location in this decision situation.

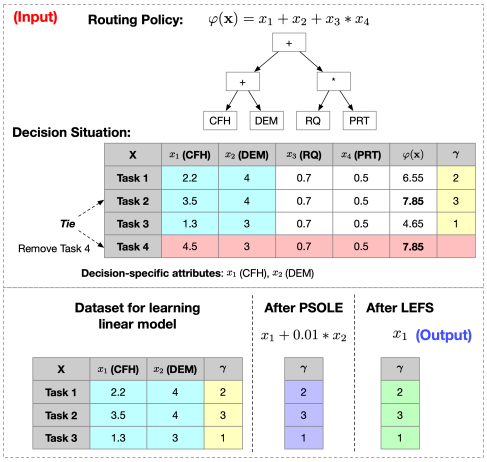


Fig. 4. An example of the local ranking explanation.

Algorithm 4: Global Explanation Method

Input: A set of local explanations $\langle \mathbf{X}_1, \boldsymbol{\beta}_1 \rangle, \ldots, \langle \mathbf{X}_L, \boldsymbol{\beta}_L \rangle$

$(\mathcal{B}_1, \ldots, \mathcal{B}_K) = \texttt{kmeans}(\boldsymbol{\beta}_1, \ldots, \boldsymbol{\beta}_L)$;
for $i = 1 \to K$ **do** Set $\mathcal{X}_i = \emptyset$;
for $i = 1 \to L$ **do**
  Find the cluster index c_i of $\boldsymbol{\beta}_i$;
  Add the first row of $\mathbf{X}_i^{\langle global \rangle}$ into $\mathcal{X}_{c_i}$;
end
Explain the global behaviour based on $\langle \mathcal{X}_1, \mathcal{B}_1 \rangle, \ldots, \langle \mathcal{X}_K, \mathcal{B}_K \rangle$;

B. Global Explanation

Intuitively, the global behaviour of a routing policy contains its local behaviour in each possible decision situation. Therefore, we proposed a global explanation method based on a collection of various decision situations and their corresponding local explanations, which is described in Algorithm 4. Given a large number of L decision situations $\mathbf{X}_i$ and their local explanations (i.e., the linear models $\boldsymbol{\beta}_i$), We first cluster the decision situations into a small number of K clusters using K-means so that each cluster contains similar local explanations (i.e., coefficients of the linear models). Then, we analyse the distribution of decision-independent attributes (that represent the global environment) of the decision situations in each cluster. Finally, we can explain the global behaviour of the routing policy based on its local behaviours in each cluster (e.g., in the format of "if (the environment (decision-independent features) is in cluster k), then (the local explanation for cluster k)").

IV. Experimental Studies

In this section, we verify the proposed LGRE method on the commonly used UCARP datasets and promising routing

TABLE I
THE GP TERMINAL SET WITH LOCAL (L) AND GLOBAL (G) ATTRIBUTES.

Terminal	G/L	Description
SC	L	cost of serving the candidate task
CFD	L	cost from the depot to the candidate task
CTT1	L	cost from the candidate to its closest remaining task
CTD	L	cost from the candidate task to the depot
CFH	L	cost from the current vehicle location to the candidate task
DEM	L	expected demand of the candidate task
DEM1	L	demand of the closest unserved task to the candidate task
CR	G	cost to refill, i.e., from the current location to the depot
FULL	G	fullness (served demand over capacity) of the vehicle
PRT	G	percentage of unserved tasks
RQ	G	remaining capacity of the vehicle
ERC	G	a random constant value

policies evolved by GP. Specifically, we use the Ugdb and Uval datasets [1], [14] containing 57 instances ranging from 22 to 97 tasks.

For each UCARP instance, we run a standard GP and select the routing policy with the best test performance to be explained. The terminal (attribute) set of GP is shown in Table I, where the local (global) attributes are shown as "L" ("G"). All decision-specific attributes are stochastic because all decision-specific attributes are calculated based on stochastic deadheading cost and stochastic demand. The function set is set to $\{+, -, \times, /, \min, \max\}$ where "/" operator returns 1 if divided by 0. The population size is set to 1000. The tournament selection size is set to 7. The maximal GP tree size is set to 8. The crossover and mutation rates are 0.85 and 0.15, respectively. The total number of generations is set to 100. Based on previous studies [7], [14], such a GP setting can evolve a promising routing policy. However, they are too large (e.g., with more than 50 nodes and complex combinations) and hard to interpret directly.

For each UCARP instance and its GP-evolved routing policy to be explained, we run 30 simulations with different random seeds (different sampled values of the random demands and deadheading costs). Each simulation consists of a number of decision situations (when a vehicle becomes idle and selects the next task to serve). Then we conduct two steps of experiments:

1) Use the *local ranking explanation* to explain the local behaviour of the routing policy in each decision situation.
2) Use the *global explanation* method to summarise the local explanations in all these decision situations into a global explanation.

For the local ranking explanation, we use the standard PSO with star topology to learn the linear model in Algorithm 2. We set 200 particles and 200 iterations. The parameters c_1 and c_2 are set to 1.49618. The inertia weight w is set to 0.729843788 [66]–[68].

For the global explanation, we use the K-means method with $K = 5$ to cluster the linear models for all the instances.

A. Local Explanation Results

First we investigate the accuracy of the local explanations in terms of consistency and correlation, and the complexity of the learned linear models in terms of number of features used (with non-zero coefficients). To verify the effectiveness of the proposed model learning method (Algorithms 2 and 3), we compare with the Lasso regression method [57], which considers both regression error and feature selection (i.e., better interpretability) in the model learning. Specifically, to fit the rank vector $\boldsymbol{\gamma}$, it minimises the following objective function:

$$\min_{\boldsymbol{\beta}} \frac{1}{M} ||\boldsymbol{\gamma} - \mathbf{X}^{\langle local \rangle}\boldsymbol{\beta}||_2^2 + \lambda||\boldsymbol{\beta}||_1. \quad (4)$$

Note that Lasso can only use the raw linear aggregation $\mathbf{X}^{\langle local \rangle}\boldsymbol{\beta}$ instead of the converted ranks, since it requires continuity and smooth gradient.

Tables II and III show the mean and standard deviation of the consistency, correlation and number of used features obtained by Lasso and the proposed LRE over all the decision situations of the 30 simulations for the Ugdb and Uval dataset, respectively.

In terms of accuracy, we clearly see that the proposed LRE achieved much higher consistency and correlation than Lasso. We conducted Wilcoxon rank sum test with significance level of 0.05, and the results show that LRE significantly outperformed Lasso for all the instances in terms of both consistency and correlation. Furthermore, LRE managed to consistently achieve consistency of 1.0 (i.e., always select the same task as the explained routing policy) for 16 out of the 23 Ugdb instances and 24 out of the 34 Uval instances. Overall, the mean consistency is no worse than 0.91 (Ugdb5). LRE also achieved very high correlation, which is as high as 0.98 in most cases. There are only 3 Ugdb instances where LRE achieved less than 0.9 correlation. In summary, the linear models learned by LRE can not only make the same decision (select the same task), but also achieve the same preference relationship among the candidate tasks as the explained routing policy. This verifies the effectiveness of LRE.

In comparison, Lasso obtained much lower consistency (mostly ranging from 0.6 to 0.8) and correlation (mostly between 0.9 and 0.98) than LRE. The consistency is much lower than correlation. This is because Lasso treats the tasks equally important, and cannot focus on a particular task selected by the routing policy. By minimising the mean square error, Lasso can achieve high correlation but makes more mistakes on the selected tasks. The advantage of LRE over Lasso, especially in consistency, verifies the effectiveness of the new fitness function in the proposed $\texttt{PSOLE}(\cdot)$ algorithm that considers consistency and correlation separately.

In terms of number of features (interpretability), we can see that both LRE and Lasso can obtain interpretable linear models with a small number of features (less than 4 features in almost all the cases). Lasso obtained much fewer features than LRE for some instances (e.g., Ugdb15 and Ugdb17). However, this is at the cost of lower accuracy (i.e., consistency and correlation). LRE obtained fewer features than Lasso on a number of Uval instances, which are larger and more complex than the Ugdb instances. This is because LRE converts the priority values to the ranks, and is more tolerant to the raw priority values. In other words, a simple model that

produces very different priority values but still converts to the same ranks can still achieve high consistency and correlation. However, Lasso minimises the error based on the raw priority values, thus is much harder to achieve high consistency and correlation with a simple model.

TABLE II
THE MEAN AND STANDARD DEVIATION FOR THREE METRICS OF IDENTIFYING THE PERFORMANCE OF THE LOCAL EXPLANATION OVER 30 SIMULATIONS ON *Ugdb* DATASET.

	Consistency		Correlation		Complexity	
Instance	Lasso	LRE	Lasso	LRE	Lasso	LRE
Ugdb1	0.9(0.03)	1.0(0.0)	0.97(0.004)	0.99(0.001)	3.1(0.0)	3.0(0.1)
Ugdb2	0.6(0.02)	1.0(0.0)	0.94(0.004)	0.94(0.003)	3.6(0.1)	3.0(0.1)
Ugdb3	0.8(0.05)	1.0(0.02)	0.97(0.004)	0.99(0.002)	2.5(0.1)	1.5(0.1)
Ugdb4	0.7(0.04)	0.9(0.0)	0.98(0.011)	0.99(0.001)	3.4(0.2)	2.8(0.2)
Ugdb5	0.6(0.02)	0.9(0.03)	0.55(0.036)	0.76(0.007)	3.3(0.1)	3.5(0.2)
Ugdb6	0.6(0.02)	1.0(0.0)	0.87(0.006)	0.89(0.018)	3.8(0.1)	3.6(0.1)
Ugdb7	0.9(0.02)	1.0(0.0)	0.77(0.02)	0.94(0.004)	3.3(0.0)	3.4(0.1)
Ugdb8	0.7(0.04)	1.0(0.0)	0.94(0.023)	1.0(0.0)	2.6(0.1)	2.2(0.1)
Ugdb9	0.7(0.04)	1.0(0.01)	0.92(0.012)	0.99(0.001)	1.7(0.1)	1.3(0.0)
Ugdb10	0.7(0.02)	1.0(0.0)	0.95(0.002)	0.98(0.001)	2.0(0.0)	1.8(0.1)
Ugdb11	0.7(0.05)	1.0(0.0)	0.97(0.003)	0.99(0.001)	3.4(0.1)	3.6(0.1)
Ugdb12	0.9(0.04)	1.0(0.0)	0.96(0.004)	0.98(0.002)	3.9(0.1)	3.4(0.2)
Ugdb13	0.5(0.08)	0.9(0.03)	0.91(0.021)	0.99(0.005)	2.7(0.1)	2.5(0.1)
Ugdb14	0.6(0.05)	1.0(0.0)	0.96(0.015)	0.99(0.002)	2.3(0.1)	3.0(0.1)
Ugdb15	0.9(0.0)	1.0(0.0)	0.67(0.009)	0.98(0.0)	1.0(0.0)	3.7(0.0)
Ugdb16	0.8(0.01)	1.0(0.0)	0.87(0.003)	0.98(0.001)	2.2(0.1)	3.5(0.1)
Ugdb17	0.8(0.0)	1.0(0.0)	0.76(0.008)	0.99(0.003)	1.7(0.0)	3.1(0.0)
Ugdb18	0.8(0.01)	1.0(0.0)	0.83(0.008)	1.0(0.0)	2.5(0.0)	2.9(0.1)
Ugdb19	0.6(0.06)	1.0(0.0)	0.91(0.018)	1.0(0.0)	2.5(0.2)	3.7(0.4)
Ugdb20	0.8(0.0)	1.0(0.0)	0.97(0.001)	0.99(0.0)	2.7(0.0)	2.9(0.2)
Ugdb21	0.8(0.02)	1.0(0.0)	0.94(0.013)	0.98(0.001)	3.0(0.1)	3.6(0.1)
Ugdb22	0.7(0.04)	1.0(0.0)	0.82(0.039)	0.98(0.002)	3.0(0.1)	4.0(0.1)
Ugdb23	0.7(0.03)	1.0(0.0)	0.7(0.038)	0.86(0.011)	2.2(0.0)	2.9(0.1)

Overall, it is clear that for almost all the decision situations in the experiment, the proposed LRE can successfully learn simple and interpretable linear models (with less than 4 features) that behave almost the same as the explained routing policy in terms of consistency and correlation.

B. Case Studies on Local Explanations

In this section, we provide examples of the local explanations in different decision situations. Specifically, we consider the following scenarios:

- Small-sized instance (Ugdb2 with 26 tasks),
- Medium-sized instance (Ugdb9 with 51 tasks),
- Large-sized instance (Uval10C with 97 tasks),

For each instance, we consider decision situations where the vehicle is (a) empty; (b) half-loaded; or (c) full.

1) Ugdb2: The GP-evolved routing policy for Ugdb2 has the test performance of 361.78 (vs 414.50 of the path scanning heuristic [5]). It contains 55 nodes and 9 unique features.

In the first decision situation, the decision-independent feature FULL $= 0$, i.e., the vehicle is completely empty. The linear model obtained by LRE is as follows. Its consistency is 1, and correlation is 0.97.

$$\text{priority}^{\text{Ugdb2}}_{\text{empty}} = 0.1 \cdot \text{CFD} + 0.46 \cdot \text{CFH} - 0.07 \cdot \text{CTD} - \text{SC}. \quad (5)$$

Explanation. *Note that when the vehicle is empty, it must be at the depot. In this case, CFD and CFH are essentially the same. From the linear model, the vehicle prefers the tasks close to it (*$0.1 \cdot$*CFD*$+0.46 \cdot$*CFH) and have large serving costs (*$-$*SC). It also slightly prefers the tasks whose tail nodes are far away from the depot (*$-0.07 \cdot$*CTD). This is consistent with our intuition that when a vehicle is empty, its route should be in an outgoing direction, and the closer tasks are more preferred to minimise the total cost. Furthermore, the vehicle prefers the heavier tasks with larger serving costs, since leaving them to the end of the route can lead to large recourse cost if a route failure occurs then.*

TABLE III
THE MEAN AND STANDARD DEVIATION FOR THREE METRICS OF IDENTIFYING THE PERFORMANCE OF THE LOCAL EXPLANATION OVER 30 SIMULATIONS ON *Uval* DATASET.

	Consistency		Correlation		Complexity	
Instance	Lasso	LRE	Lasso	LRE	Lasso	LRE
Uval1A	0.8(0.0)	1.0(0.0)	0.95(0.0)	0.99(0.0)	3.3(0.0)	3.3(0.1)
Uval1B	0.5(0.01)	1.0(0.0)	0.85(0.001)	0.91(0.001)	3.2(0.0)	3.2(0.1)
Uval1C	0.9(0.03)	1.0(0.0)	0.97(0.006)	1.0(0.001)	4.0(0.1)	3.9(0.1)
Uval2A	0.7(0.0)	1.0(0.0)	0.98(0.0)	0.99(0.0)	3.9(0.0)	2.0(0.1)
Uval2B	0.8(0.02)	1.0(0.01)	0.98(0.005)	0.99(0.001)	2.7(0.1)	1.6(0.1)
Uval2C	0.6(0.07)	1.0(0.01)	0.92(0.009)	0.97(0.003)	4.3(0.1)	4.1(0.1)
Uval3A	1.0(0.0)	1.0(0.0)	0.99(0.0)	1.0(0.0)	2.2(0.0)	1.4(0.1)
Uval3B	0.9(0.01)	1.0(0.0)	0.87(0.006)	0.95(0.001)	3.3(0.0)	3.8(0.1)
Uval3C	0.8(0.04)	1.0(0.0)	0.92(0.013)	0.99(0.001)	3.3(0.2)	3.3(0.2)
Uval4A	0.8(0.03)	1.0(0.0)	0.99(0.002)	1.0(0.0)	3.1(0.1)	1.9(0.1)
Uval4B	0.7(0.02)	1.0(0.0)	0.96(0.006)	0.97(0.003)	4.6(0.1)	2.7(0.2)
Uval4C	0.8(0.02)	1.0(0.0)	0.97(0.005)	0.98(0.002)	3.8(0.1)	2.7(0.1)
Uval4D	0.8(0.03)	1.0(0.0)	0.96(0.003)	1.0(0.0)	4.1(0.1)	2.8(0.1)
Uval5A	0.7(0.01)	1.0(0.01)	0.95(0.001)	0.99(0.001)	5.4(0.1)	3.1(0.1)
Uval5B	0.8(0.04)	1.0(0.0)	0.99(0.001)	1.0(0.0)	3.2(0.1)	1.2(0.1)
Uval5C	0.8(0.03)	1.0(0.0)	0.96(0.007)	0.98(0.003)	4.1(0.1)	2.8(0.1)
Uval5D	0.6(0.03)	1.0(0.0)	0.97(0.002)	0.99(0.0)	3.6(0.1)	2.3(0.1)
Uval6A	0.6(0.0)	1.0(0.0)	0.97(0.0)	0.99(0.0)	4.6(0.0)	3.2(0.1)
Uval6B	0.7(0.0)	1.0(0.0)	0.92(0.0)	0.94(0.0)	3.4(0.0)	3.2(0.1)
Uval6C	0.6(0.04)	0.9(0.02)	0.93(0.011)	0.97(0.004)	3.2(0.1)	2.3(0.1)
Uval7A	0.8(0.0)	1.0(0.0)	0.98(0.0)	1.0(0.0)	3.9(0.0)	2.3(0.1)
Uval7B	0.7(0.01)	1.0(0.0)	0.98(0.0)	1.0(0.0)	3.5(0.0)	2.3(0.1)
Uval7C	0.7(0.03)	1.0(0.0)	0.96(0.012)	0.99(0.001)	2.9(0.1)	2.3(0.1)
Uval8A	0.7(0.01)	1.0(0.0)	0.95(0.003)	0.97(0.002)	4.3(0.0)	2.3(0.1)
Uval8B	0.8(0.04)	1.0(0.01)	0.98(0.006)	0.99(0.003)	4.3(0.1)	2.1(0.2)
Uval8C	0.7(0.05)	1.0(0.0)	0.97(0.003)	0.99(0.001)	4.4(0.1)	2.8(0.1)
Uval9A	0.7(0.01)	1.0(0.0)	0.99(0.0)	1.0(0.0)	3.7(0.0)	1.5(0.0)
Uval9B	0.8(0.03)	1.0(0.0)	0.99(0.004)	1.0(0.001)	3.0(0.1)	1.3(0.1)
Uval9C	0.8(0.02)	1.0(0.01)	0.98(0.008)	0.98(0.003)	1.9(0.0)	1.3(0.0)
Uval9D	0.7(0.03)	1.0(0.0)	0.98(0.004)	0.99(0.001)	3.4(0.1)	1.7(0.1)
Uval10A	0.9(0.01)	1.0(0.0)	1.0(0.001)	1.0(0.0)	3.5(0.0)	1.2(0.0)
Uval10B	0.8(0.02)	1.0(0.0)	0.99(0.002)	1.0(0.0)	3.1(0.1)	1.2(0.0)
Uval10C	0.7(0.03)	1.0(0.01)	0.94(0.005)	0.98(0.002)	3.7(0.1)	2.6(0.1)
Uval10D	0.7(0.04)	1.0(0.0)	0.96(0.005)	0.98(0.002)	4.1(0.1)	2.7(0.1)

In the second decision situation, FULL $= 0.58$, i.e., the vehicle is a bit over half-loaded. The linear model is as follows. Its consistency is 1, and correlation is 0.97.

$$\text{priority}^{\text{Ugdb2}}_{\text{half}} = \text{CFH} + 0.1 \cdot \text{CTD} + 0.03 \cdot \text{SC}. \quad (6)$$

Explanation. *The vehicle prefers the tasks close to it (CFH) and with small serving cost (*$0.03 \cdot$*SC), and the emphasis on serving cost decreases as the vehicle becomes more loaded. However, it prefers the tasks whose tail node is closer to the depot (*$0.1 \cdot$*CTD) now. This is because after half-loaded, the vehicle should be on the way back to the depot.*

In the last decision situation, FULL $= 0.71$. The linear model is shown below. Its consistency is 1, and correlation is

0.99.

$$\text{priority}_{\text{full}}^{\text{Ugdb2}} = 0.6 \cdot \text{CFD} + \text{CFH} + 0.33 \cdot \text{CTD}. \quad (7)$$

Explanation. *When the vehicle is nearly full, the serving cost becomes redundant, and the vehicle prefers the tasks close to it (CFH) and also to the depot ($0.6 \cdot$ CFD $+ 0.33 \cdot$ CTD). Compared to Eq. (6), the coefficient of CTD is smaller in Eq. (7), but the coefficient of CFD is larger, which indicates that although the vehicle is concerned with similar characteristics after the vehicle is half-loaded. The focus on CTD and CFD vary according to the vehicle load.*

Overall, the local explanations of the three different decision situations show that the routing policy for Ugdb2 always prefers the tasks close to the vehicle. In addition, the serving cost and the costs from and to the depot play different roles in different planning stages. All the patterns are consistent with our intuition.

2) Ugdb9: The GP-evolved routing policy for Ugdb9 has the test performance of 357.15 (vs 419.23 of the path scanning heuristic). It has 79 nodes and 6 unique features.

We select three decision situations for Ugdb9 with different vehicle loads. The first decision situation has Full $= 0$, i.e., the vehicle is completely empty. The linear model is as follows. Its consistency is 1, and correlation is 1.

$$\text{priority}_{\text{empty}}^{\text{Ugdb9}} = 0.65 \cdot \text{CFH} - \text{CTD}. \quad (8)$$

Explanation. *The vehicle simply prefers the tasks close to it ($0.65 \cdot$ CFH) and with tail nodes far away from the depot ($-$CTD). Both features are very important for the decision, since their coefficients are both large.*

In the second decision situation, FULL $= 0.49$, i.e., the vehicle is half-loaded. The linear model is shown as follows. Its consistency is 1, and correlation is 0.99.

$$\text{priority}_{\text{half}}^{\text{Ugdb9}} = \text{CFH}. \quad (9)$$

Explanation. *This is a surprisingly simply linear model. The vehicle simply selects the task that is closest to it (ties are broken by selecting the one with the lowest index). This indicates that for Ugdb9, the nearest neighbour heuristic can work just well in the middle of the routing stage.*

The last decision situation has FULL $= 0.93$, i.e., the vehicle is almost full. The corresponding linear model is shown below. Its consistency is 1, and correlation is 1.

$$\text{priority}_{\text{full}}^{\text{Ugdb9}} = 0.46 \cdot \text{CFH} + \text{CTD}. \quad (10)$$

Explanation. *The vehicle prefers the tasks close to it ($0.46 \cdot$ CFH and also close to the depot CTD. This is consistent with our intuition, since when the vehicle is almost full, it should go back to the depot, and serve the tasks on its way back. Compared with the model Eq. (7) for Ugdb2, the cost to the depot is more important than the cost from the vehicle (CTD has a larger coefficient). This could be due to the different graph topology between Ugdb2 and Ugdb9.*

Overall, the linear models obtained for the three different decision situations for Ugdb9 are also easy to interpret, and their patterns are consistent with our intuition.

3) Uval10C: The GP-evolved routing policy for Uval10C has the test performance of 480.36 (vs 504.44 of the path scanning heuristic). It contains 59 nodes and 8 unique features.

For Uval10C, the first decision situation has FULL $= 0$, i.e., the vehicle is completely empty. The linear model is shown below. Its consistency is 1, and correlation is 0.95.

$$\text{priority}_{\text{empty}}^{\text{Uval10C}} = 0.12 \cdot \text{CFD} + 0.82 \cdot \text{CFH} - \text{CTD}. \quad (11)$$

Explanation. *Note that CFD $=$ CFH when vehicle is empty. Thus, the vehicle prefers the tasks close to it $0.82 \cdot$ CFH and tail nodes far away from the depot ($-$CTD). It also slightly prefers the tasks whose head node is close to the depot (where the vehicle is when empty). This pattern is similar to that in the linear models for Ugdb2 Eq. (5) and Ugdb9 Eq. (8) when the vehicle is empty, and is consistent with our intuition.*

The second decision has FULL $= 0.47$, i.e., the vehicle is a bit less than half-loaded. The linear model is as follows. Its consistency is 1, and correlation is 0.99.

$$\text{priority}_{\text{half}}^{\text{Uval10C}} = -0.53 \cdot \text{CFD} + \text{CFH}. \quad (12)$$

Explanation. *The vehicle prefers the tasks close to it (CFH) and whose tail nodes are far away from the depot ($-0.53 \cdot$ CFD). The preference on small CFH is consistent with the other cases and our intuition. The preference on large CFD when half-loaded, although not considered in other cases, can be explained if the graph contains more tasks that are far away from the depot. This suggests that the vehicle still needs to travel outwardly when half loaded.*

In the last decision situation, FULL $= 0.96$, which means that the vehicle is almost full. The corresponding linear model is shown as follows. Its consistency is 1, and correlation is 1.

$$\text{priority}_{\text{full}}^{\text{Uval10C}} = \text{CTD}. \quad (13)$$

Explanation. *This linear model is surprisingly simple. It simply selects the tasks whose tail node is closest to the depot. This makes sense since toward the end of a route, it is highly likely to have route failures, and tasks with small CTD can reduce the recourse cost. Interestingly, the vehicle does not even consider CFH, which is known to be the most important routing feature. In other words, this local explanation is specific to this decision situation where all the candidate tasks have the same CFH. It also suggests the need for a more general global explanation.*

Overall, we found some different patterns for the local explanations of Uval10C from that of Ugdb2 and Ugdb9. However, these patterns are explainable and dependent on the instance characteristics such as graph topology. The local explanations seem to be over-specific to instance and decision situation. Therefore, it is desirable to have a more general global explanation that is independent of decision situation.

C. Case Studies on Global Explanations

In this section, we show two examples of the global explanations obtained by Algorithm 4 (with $K = 5$) for Ugdb2 and Uval10C. Specifically, for each instance, we first collect the

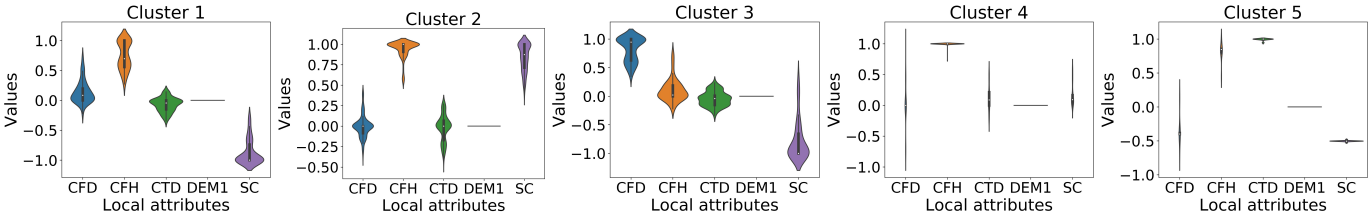


Fig. 5. The distribution of the the coefficients of decision-specific attributes for each cluster in Ugdb2.

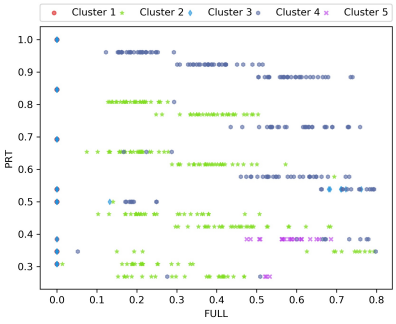


Fig. 6. The distribution of decision-independent attributes of the decision situations in each cluster of Ugdb2.

local explanations (linear models) of all the decision situations encountered in the 30 simulations (excluding those with only a single candidate task). Then, we cluster the $\boldsymbol{\beta}$ vectors of the local explanations into 5 clusters. The presence of too many clusters in the global explanation would make it difficult for the user to understand the whole global explanation. However, too few clusters can again lead to a lack of clarity in the global explanation. Here, to retain good interpretability, we limit the maximal number of clusters to 5. We have tried 2, 3, 4, 5 clusters and found that 5 clusters showed the best clustering performance on $\boldsymbol{\beta}$. Then, we observe the distribution of the decision-independent attributes in each cluster and explain the relationship between the decision-independent attribute distribution (decision situation properties) and the $\boldsymbol{\beta}$ vector (routing policy behaviour).

1) Ugdb2: Fig. 5 shows the distribution (violin plot) of the $\boldsymbol{\beta}$ vectors of each cluster in Ugdb2. We can see that different clusters have different distributions of the $\boldsymbol{\beta}$ vectors. In each cluster, the relative importance (coefficient magnitude) of different decision-specific attributes are very clear. For example, in cluster 1, CFH and SC are much more important than others (their coefficients have much larger magnitude).

Fig. 6 shows the distribution of the decision-independent attributes of the decision situations in each cluster of Ugdb2, where each point stands for a decision situation. Note that there are four decision-independent attributes: CR, FULL, PRT and RQ. In the figure, we show only two decision-independent attributes FULL and PRT, and omit CR and RQ to facilitate visualisation. By definition, we have $\text{RQ} = (1 - \text{FULL}) \cdot Q$, thus RQ is redundant if FULL is already shown. Second, CR is omitted because it has essentially no effect on the clustering results. Other decision-independent attributes are sufficient to represent a decision situation. In addition, it is easier to observe the clustering results in two dimensions.

From Figs. 5 and 6, we can see the following patterns of the routing policy for Ugdb2.

- **(Cluster 1.)** The cluster 1 contains the red circle points on the left of Fig. 6 with $\text{FULL} = 0$, which are overlapped with the blue points (cluster 3). ***IF*** the vehicle is empty, ***THEN*** the vehicle tends to select the tasks close to its current location (large positive coefficient of CFH) and large serving cost (large negative coefficient of SC). It slightly prefers the tasks whose head nodes are close to the depot (slight positive coefficient of CFD) and tail nodes are far away from the depot (slight negative coefficient of CTD), i.e., the tasks with outward directions. However, the importance of CFD and CTD are much less than CFH and SC. DEM1 is completely redundant, as its coefficient is always 0.
- **(Cluster 2.)** ***IF*** toward the later service process ($\text{PRT} < 0.5$) or in the early stage ($0.5 < \text{PRT} < 0.8$) and the vehicle is less than half full ($\text{FULL} < 0.5$, ***THEN*** the vehicle prefers the tasks close to it (large positive coefficient of CFH) and with small serving cost (large positive coefficient of SC). All the other decision-specific attributes are much less important, as their coefficients are close to 0.
- **(Cluster 3.)** This points in cluster 3 heavily overlap with that in cluster 1 in Fig. 6 with $\text{FULL} = 0$. By comparing the decision-specific attribute distributions of these two clusters in Fig. 5, we can see that the clusters 1 and 3 have similar distributions, except that the coefficients of CFD and CFH are swapped. Note that when the vehicle is empty, it is essentially at the depot, and CFD is essentially equal to CFH. Therefore, these two clusters essentially represent the same situation, i.e., ***IF*** the vehicle is empty and at the depot, ***THEN*** the vehicle prefers the tasks close to it and with a large serving cost.
- **(Cluster 4.)** ***IF*** at the very beginning of the service process ($\text{PRT} > 0.9$) or during the first half of the service process ($0.5 < \text{PRT} < 0.9$) and the vehicle is more than half full ($\text{FULL} > 0.5$), ***THEN*** the vehicle highly prefers the tasks close to it (largest positive coefficient of CFH). The importance of other decision-specific attributes are almost negligible (their coefficients are close to 0).
- **(Cluster 5.)** ***IF*** in the later stage of the service process ($\text{PRT} < 0.5$) and when the vehicle is substantially loaded ($0.5 < \text{FULL} < 0.7$), ***THEN*** the vehicle tends to select

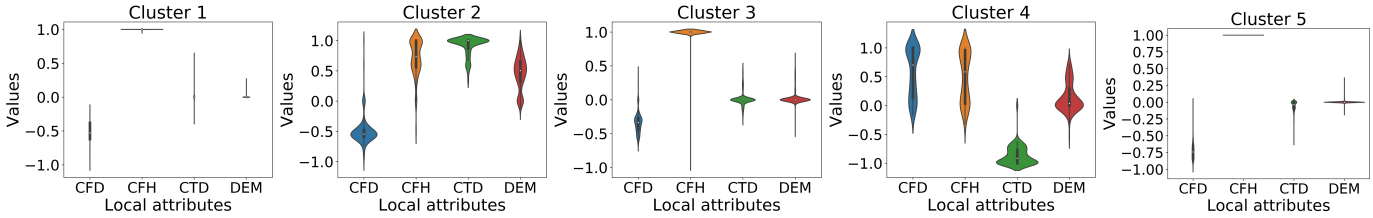


Fig. 7. The distribution of the coefficients of **decision-specific attributes** for each cluster in **Uval10C**.

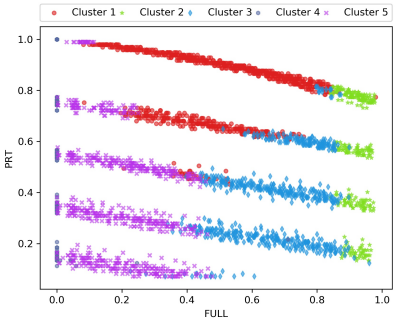


Fig. 8. The distribution of **decision-independent attributes** of the decision situations in each cluster of **Uval10C**.

the close tasks (large positive CFH coefficient) and close to the depot (large positive CTD coefficient). In addition, it prefers the tasks whose head nodes are far away from the depot (its direction is towards the depot) and with small serving cost (small recourse cost if route failure occurs).

Overall, the global explanation of the routing policy for Ugdb2 is as follows. The vehicle always prefers the tasks close to it. When the vehicle is empty, it prefers the tasks withe outward directions from the depot and large serving cost. However, toward the later stage of the service process and when the vehicle is more loaded, the vehicle gradually switches its focus on the tasks with smaller serving cost, and whose directions are towards the depot.

2) Uval10C: Fig. 7 shows the distribution of the decision-specific attribute coefficients for each cluster in Uval10C. Note that the routing policy for Uval10C is different from that for Ugdb2, and contains a different set of decision-specific attributes. Specifically, the routing policy for Uval10C considers DEM, while ignores DEM1 and SC. The corresponding distribution of decision-independent attributes (FULL and PRT) of the decision situations in each cluster of Uval10C is shown in Fig. 8.

From the figures, we can observe the following patterns.

- **(Cluster 1.)** ***IF*** in the early stage of the service process (PRT > 0.6, ***THEN*** the vehicle prefers the tasks close to it (coefficient of CFH close to 1) and whose head node is far away from the depot (negative CFD coefficient). This indicates that the vehicle aims to travel to the regions far away from the depot as early as possible.
- **(Cluster 2.)** ***IF*** the vehicle is close to full (route failures are more likely to occur), ***THEN*** the vehicle tends to select the task close to it (positive CFH coefficient), whose directions are towards the depot (negative CFD coefficient and positive CTD coefficient), and with small expected demand (positive DEM coefficient) to reduce the likelihood of route failures.
- **(Cluster 3.)** ***IF*** the vehicle is substantially loaded but not very full ($0.4 <$ FULL < 0.8), ***THEN*** the vehicle highly focuses on the tasks close to it (CFH coefficient close to 1), and slightly considers the tasks whose head nodes are far away from the depot (slightly negative CFD coefficient).
- **(Cluster 4.)** ***IF*** the vehicle is empty and at the depot, ***THEN*** the vehicle tends to select the tasks close to it (positive coefficients for CFH and CFD) and whose directions are outward from the depot (positive CFD coefficient and negative CTD coefficient).
- **(Cluster 5.)** ***IF*** the vehicle is less than half loaded, ***THEN*** the vehicle prefers the task close to it (CFH coefficient is 1) and whose head nodes are far away from the depot (negative CFD coefficient).

Overall, we can give the following global explanation of the routing policy for Uval10C. The vehicle always prefers close tasks. When the vehicle is empty and at the depot, it prefers the tasks with outward directions from the depot. When there are many unserved tasks and the vehicle is at the beginning of its route, it aims to travel to the regions far away from the depot as early as possible. Then, towards the later stage of the route, the vehicle prefers the tasks pointing towards the depot and with small demand to reduce possible route failures.

The results and analysis showed that the new proposed approach can get better consistency, correlation and complexity, which indicates that it can better explain the corresponding routing policy than the approach in [62]. In addition, the new proposed approach not only provides local explanations for the corresponding routing policy but can also provide a global explanation for users to understand the whole routing policy, which cannot be achieved by the work in [62].

V. Conclusions

This paper aims to explain the complex GP-evolved routing policies for UCARP. This goal has been successfully achieved by the proposed local-global ranking explanation method. The local ranking explanation can obtain a simple linear model that accurately explains the ranking behaviour of the routing policy in each decision situation. The global explanation method can summarise the local linear models into an if-then explanation.

We provide a number of case studies to show how a complex GP-evolved routing policy for UCARP can be explained by the proposed method. Our method is not restricted to UCARP, but can be easily generalised to other optimisation and machine learning problems where decisions are made based on ranking.

In the future, we will consider use GP to evolve both linear and non-linear models as the local explanations for decision situations that are easy and hard to fit, respectively. Furthermore, it will be promising to employ this local-global layered framework to develop intrinsic explainable GP approaches such as by grammar-guided GP. Last but not least, although proposed for UCARP, the LGRE method is generic and can be easily extended to other sequential decision making problems, where each decision is made by ranking the candidate actions, such as dynamic scheduling [25], [69], [70] and reinforcement learning tasks [21].

REFERENCES

[1] Y. Mei, K. Tang, and X. Yao, "Capacitated arc routing problem in uncertain environments," in *Proceeding of IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2010, pp. 1–8.

[2] B. L. Golden and R. T. Wong, "Capacitated arc routing problems," *Networks*, vol. 11, no. 3, pp. 305–315, 1981.

[3] H. Handa, L. Chapman, and X. Yao, "Robust route optimization for gritting/salting trucks: a cercia experience," *IEEE Computational Intelligence Magazine*, vol. 1, no. 1, pp. 6–9, 2006.

[4] S. Amponsah and S. Salhi, "The investigation of a class of capacitated arc routing problems: The collection of garbage in developing countries," *Waste Management*, vol. 24, no. 7, pp. 711–721, 2004.

[5] P. Lacomme, C. Prins, and W. Ramdane-Cherif, "Competitive memetic algorithms for arc routing problems," *Annals of Operations Research*, vol. 131, no. 1-4, pp. 159–185, 2004.

[6] M. R. Khouadjia, B. Sarasola, E. Alba, L. Jourdan, and E.-G. Talbi, "A comparative study between dynamic adapted pso and vns for the vehicle routing problem with dynamic requests," *Applied Soft Computing*, vol. 12, no. 4, pp. 1426–1439, 2012.

[7] Y. Liu, Y. Mei, M. Zhang, and Z. Zhang, "Automated heuristic design using genetic programming hyper-heuristic for uncertain capacitated arc routing problem," in *Proceeding of GECCO*. ACM, 2017, pp. 290–297.

[8] U. Ritzinger, J. Puchinger, and R. F. Hartl, "A survey on dynamic and stochastic vehicle routing problems," *International Journal of Production Research*, vol. 54, no. 1, pp. 215–231, 2016.

[9] Y. Mei and M. Zhang, "Genetic programming hyper-heuristic for multi-vehicle uncertain capacitated arc routing problem," in *Proceeding of GECCO*. ACM, 2018, pp. 141–142.

[10] T. Miller, "Explanation in artificial intelligence: Insights from the social sciences," *Artificial intelligence*, vol. 267, pp. 1–38, 2019.

[11] C. Molnar, *Interpretable Machine Learning*. Lulu. com, 2020.

[12] D. V. Carvalho, E. M. Pereira, and J. S. Cardoso, "Machine learning interpretability: A survey on methods and metrics," *Electronics*, vol. 8, no. 8, p. 832, 2019.

[13] S. Wang, Y. Mei, and M. Zhang, "Novel ensemble genetic programming hyper-heuristics for uncertain capacitated arc routing problem," in *Proceeding of GECCO*, 2019, pp. 1093–1101.

[14] S. Wang, Y. Mei, M. Zhang, and X. Yao, "Genetic programming with niching for uncertain capacitated arc routing problem," *IEEE Transactions on Evolutionary Computation*, vol. 26, no. 1, pp. 73–87, 2022.

[15] J. Branke, T. Hildebrandt, and B. Scholz-Reiter, "Hyper-heuristic evolution of dispatching rules: A comparison of rule representations," *Evolutionary computation*, vol. 23, no. 2, pp. 249–277, 2015.

[16] Y.-H. Jia, Y. Mei, and M. Zhang, "Learning heuristics with different representations for stochastic routing," *IEEE Transactions on Cybernetics*, pp. 1–15, 2022.

[17] Z. C. Lipton, "The mythos of model interpretability," *Queue*, vol. 16, no. 3, pp. 31–57, 2018.

[18] F. Zhang, Y. Mei, S. Nguyen, M. Zhang, and K. C. Tan, "Surrogate-assisted evolutionary multitask genetic programming for dynamic flexible job shop scheduling," *IEEE Transactions on Evolutionary Computation*, vol. 25, no. 4, pp. 651–665, 2021.

[19] B. Tan, H. Ma, Y. Mei, and M. Zhang, "A cooperative coevolution genetic programming hyper-heuristic approach for on-line resource allocation in container-based clouds," *IEEE Transactions on Cloud Computing*, pp. 1–1, 2020.

[20] R. J. Urbanowicz and W. N. Browne, *Introduction to learning classifier systems*. Springer, 2017.

[21] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.

[22] J. Liu, K. Tang, and X. Yao, "Robust optimization in uncertain capacitated arc routing problems: Progresses and perspectives," *IEEE Computational Intelligence Magazine*, vol. 16, no. 1, pp. 63–82, 2021.

[23] E. W. Dijkstra *et al.*, "A note on two problems in connexion with graphs," *Numerische mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

[24] J. Branke, S. Nguyen, C. W. Pickardt, and M. Zhang, "Automated design of production scheduling heuristics: A review," *IEEE Transactions on Evolutionary Computation*, vol. 20, no. 1, pp. 110–124, 2015.

[25] F. Zhang, Y. Mei, S. Nguyen, and M. Zhang, "Correlation coefficient based recombinative guidance for genetic programming hyper-heuristics in dynamic flexible job shop scheduling," *IEEE Transactions on Evolutionary Computation*, vol. 25, no. 3, pp. 552–566, 2021.

[26] G. Fleury, P. Lacomme, and C. Prins, "Evolutionary algorithms for stochastic arc routing problems," in *Workshops on Applications of Evolutionary Computation*. Springer, 2004, pp. 501–512.

[27] J. Wang, K. Tang, and X. Yao, "A memetic algorithm for uncertain capacitated arc routing problems," in *IEEE Workshop on MC*. IEEE, 2013, pp. 72–79.

[28] E. Babaee Tirkolaee, M. Alinaghian, M. Bakhshi Sasi, and M. Seyyed Esfahani, "Solving a robust capacitated arc routing problem using a hybrid simulated annealing algorithm: a waste collection application," *Journal of Industrial Engineering and Management Studies*, vol. 3, no. 1, pp. 61–76, 2016.

[29] J. Wang, K. Tang, J. A. Lozano, and X. Yao, "Estimation of the distribution algorithm with a stochastic local search for uncertain capacitated arc routing problems," *IEEE Transactions on Evolutionary Computation*, vol. 20, no. 1, pp. 96–109, 2016.

[30] S. Tsutsui, M. Pelikan, and D. E. Goldberg, "Using edge histogram models to solve permutation problems with probabilistic model-building genetic algorithms," *IlliGAL Report*, vol. 2003022, 2003.

[31] H. Tong, L. L. Minku, S. Menzel, B. Sendhoff, and X. Yao, "A novel generalised meta-heuristic framework for dynamic capacitated arc routing problems," *arXiv preprint arXiv:2104.06585*, 2021.

[32] H. Tong, L. Minku, S. Menzel, B. Sendhoff, and X. Yao, "A hybrid local search framework for the dynamic capacitated arc routing problem," in *Proceeding of GECCO*, 2021, pp. 139–140.

[33] T. Weise, A. Devert, and K. Tang, "A developmental solution to (dynamic) capacitated arc routing problems using genetic programming," in *Proceeding of GECCO*. ACM, 2012, pp. 831–838.

[34] J. MacLachlan, Y. Mei, J. Branke, and M. Zhang, "Genetic programming hyper-heuristics with vehicle collaboration for uncertain capacitated arc routing problems," *Evolutionary computation*, pp. 1–29, 2019.

[35] Y. Liu, Y. Mei, M. Zhang, and Z. Zhang, "A predictive-reactive approach with genetic programming and cooperative coevolution for the uncertain capacitated arc routing problem," *Evolutionary computation*, vol. 28, no. 2, pp. 289–316, 2020.

[36] M. A. Ardeh, Y. Mei, M. Zhang, and X. Yao, "Knowledge transfer genetic programming with auxiliary population for solving uncertain capacitated arc routing problem," *IEEE Transactions on Evolutionary Computation*, pp. 1–1, 2022.

[37] A. Cano, A. Zafra, and S. Ventura, "An interpretable classification rule mining algorithm," *Information Sciences*, vol. 240, pp. 1–20, 2013.

[38] B. P. Evans, B. Xue, and M. Zhang, "What's inside the black-box? a genetic programming method for interpreting complex machine learning models," in *Proceedings of the Genetic and Evolutionary Computation Conference*, 2019, pp. 1012–1020.

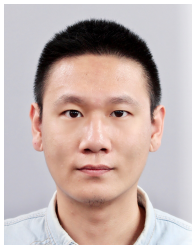
[39] L. A. Ferreira, F. G. Guimarães, and R. Silva, "Applying genetic programming to improve interpretability in machine learning models," *arXiv preprint arXiv:2005.09512*, 2020.

[40] D. Hein, S. Udluft, and T. A. Runkler, "Interpretable policies for reinforcement learning by genetic programming," *Engineering Applications of Artificial Intelligence*, vol. 76, pp. 158–169, 2018.

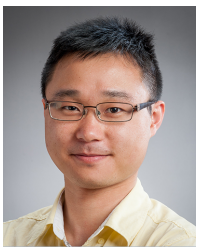
[41] F. J. Berlanga, A. Rivera, M. J. del Jesús, and F. Herrera, "Gp-coach: Genetic programming-based learning of compact and accurate fuzzy rule-based classification systems for high-dimensional problems," *Information Sciences*, vol. 180, no. 8, pp. 1183–1200, 2010.

[42] T. Hu, "Can genetic programming perform explainable machine learning for bioinformatics?" in *Genetic Programming Theory and Practice XVII*. Springer, 2020, pp. 63–77.

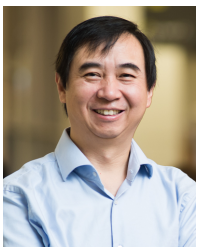
[43] H. Zhao, "A multi-objective genetic programming approach to developing pareto optimal decision trees," *Decision Support Systems*, vol. 43, no. 3, pp. 809–826, 2007.
[44] S. Wang, Y. Mei, J. Park, and M. Zhang, "Evolving ensembles of routing policies using genetic programming for uncertain capacitated arc routing problem," in *Proceeding of SSCI*. IEEE, 2019, pp. 1628–1635.
[45] ——, "A two-stage genetic programming hyper-heuristic for uncertain capacitated arc routing problem," in *Proceeding of SSCI*. IEEE, 2019, pp. 1606–1613.
[46] M. Keijzer and V. Babovic, "Dimensionally aware genetic programming," in *Proceeding of GECCO - Volume 2*. Morgan Kaufmann Publishers Inc., 1999, pp. 1069–1076.
[47] Y. Mei, S. Nguyen, and M. Zhang, "Constrained dimensionally aware genetic programming for evolving interpretable dispatching rules in dynamic job shop scheduling," in *Proceeding of SEAL*. Springer, 2017, pp. 435–447.
[48] E. Alba, C. Cotta, and J. M. Troya, "Evolutionary design of fuzzy logic controllers using strongly-typed gp," *Mathware and Soft Computing*, vol. 6, no. 1, pp. 109–124, 1999.
[49] D. Hein, S. Udluft, and T. A. Runkler, "Generating interpretable fuzzy controllers using particle swarm optimization and genetic programming," in *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, 2018, pp. 1268–1275.
[50] D. C. Hooper and N. S. Flann, "Improving the accuracy and robustness of genetic programming through expression simplification," in *Proceeding of the conference on genetic programming*, 1996, pp. 428–428.
[51] M. Zhang, P. Wong, and D. Qian, "Online program simplification in genetic programming," in *Proceeding of SEAL*. Springer, 2006, pp. 592–600.
[52] M. Zhang and P. Wong, "Genetic programming for medical classification: a program simplification approach," *Genetic Programming and Evolvable Machines*, vol. 9, no. 3, pp. 229–255, 2008.
[53] D. Kinzett, M. Zhang, and M. Johnston, "Using numerical simplification to control bloat in genetic programming," in *Proc.of SEAL*. Springer, 2008, pp. 493–502.
[54] A. Song, D. Chen, and M. Zhang, "Contribution based bloat control in genetic programming," in *Proceeding of IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2010, pp. 1–8.
[55] Y. Mei, Q. Chen, A. Lensen, B. Xue, and M. Zhang, "Explainable artificial intelligence by genetic programming: A survey," *IEEE Transactions on Evolutionary Computation*, pp. 1–1, 2022.
[56] K. Arulkumaran, A. Cully, and J. Togelius, "Alphastar: An evolutionary computation perspective," in *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, 2019, pp. 314–315.
[57] R. Tibshirani, "Regression shrinkage and selection via the lasso," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 58, no. 1, pp. 267–288, 1996.
[58] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nature Machine Intelligence*, vol. 1, no. 5, pp. 206–215, 2019.
[59] S. Wang, Y. Mei, and M. Zhang, "Two-stage multi-objective genetic programming with archive for uncertain capacitated arc routing problem," in *Proceedings of the Genetic and Evolutionary Computation Conference*, 2021, pp. 287–295.
[60] ——, "A multi-objective genetic programming approach with self-adaptive α dominance to uncertain capacitated arc routing problem," in *Proceedings of IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2021, pp. 636–643.
[61] A. B. Arrieta, N. Díaz-Rodríguez, J. Del Ser, A. Bennetot, S. Tabik, A. Barbado, S. García, S. Gil-López, D. Molina, R. Benjamins *et al.*, "Explainable artificial intelligence (xai): Concepts, taxonomies, opportunities and challenges toward responsible ai," *Information Fusion*, vol. 58, pp. 82–115, 2020.
[62] S. Wang, Y. Mei, and M. Zhang, "Local ranking explanation for genetic programming evolved routing policies for uncertain capacitated arc routing problems," in *Accepted by the ACM Genetic and Evolutionary Computation Conference (GECCO)*. ACM, jul 2022.
[63] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should i trust you? explaining the predictions of any classifier," in *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, 2016, pp. 1135–1144.
[64] R. Poli, J. Kennedy, and T. Blackwell, "Particle swarm optimization," *Swarm intelligence*, vol. 1, no. 1, pp. 33–57, 2007.
[65] H. Li, "A short introduction to learning to rank," *IEICE TRANSACTIONS on Information and Systems*, vol. 94, no. 10, pp. 1854–1862, 2011.
[66] R. C. Eberhart and Y. Shi, "Comparing inertia weights and constriction factors in particle swarm optimization," in *Proceedings of the 2000 congress on evolutionary computation. CEC00 (Cat. No. 00TH8512)*, vol. 1. IEEE, 2000, pp. 84–88.
[67] F. Van den Bergh and A. P. Engelbrecht, "A study of particle swarm optimization particle trajectories," *Information sciences*, vol. 176, no. 8, pp. 937–971, 2006.
[68] H. B. Nguyen, B. Xue, P. Andreae, and M. Zhang, "Particle swarm optimisation with genetic operators for feature selection," in *2017 IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2017, pp. 286–293.
[69] S. Nguyen, Y. Mei, and M. Zhang, "Genetic programming for production scheduling: a survey with a unified framework," *Complex & Intelligent Systems*, vol. 3, no. 1, pp. 41–66, 2017.
[70] F. Zhang, Y. Mei, S. Nguyen, K. C. Tan, and M. Zhang, "Multitask genetic programming-based generative hyperheuristics: A case study in dynamic scheduling," *IEEE Transactions on Cybernetics*, pp. 1–14, 2021.



Shaolin Wang is a fourth-year PhD student at the School of Engineering and Computer Science, Victoria University of Wellington, Wellington, New Zealand. He holds a Master of Computer Science (2019) from Victoria University of Wellington, a Graduate Diploma in Computer Science (2016) from University of Canterbury, Postgraduate Diploma in Geographic Information System (2014) from University of Otago and a BSc in Geographic Information System (2012) from Nanning Normal University. His research interests are Artificial Intelligence, Evolutionary Computation, Hyper-Heuristic, Routing and Real-World Optimisation.



Yi Mei (M'09-SM'18) received the BSc and PhD degrees from the University of Science and Technology of China, Hefei, China, in 2005 and 2010, respectively. He is an Associate Professor at the School of Engineering and Computer Science, Victoria University of Wellington, Wellington, New Zealand. His research interests include evolutionary computation and machine learning for combinatorial optimisation, genetic programming, hyper-heuristics, and explainable AI. He has over 180 fully referred publications, including the top journals in EC and Operations Research such as IEEE TEVC, IEEE TCYB, Evolutionary Computation Journal, European Journal of Operational Research, ACM Transactions on Mathematical Software. He is an Associate Editor of IEEE Transactions on Evolutionary Computation, and a guest editor of the Genetic Programming Evolvable Machine journal. He is the Founding Chair of IEEE Taskforce on Evolutionary Scheduling and Combinatorial Optimisation. He serves as a Vice-Chair of the IEEE CIS Emergent Technologies Technical Committee, and a member of Intelligent Systems Applications Technical Committee. He is a reviewer of over 50 international journals. He is a Fellow of Engineering New Zealand and an IEEE Senior Member.



Mengjie Zhang (M'04-SM'10-F'19) received the B.E. and M.E. degrees from Artificial Intelligence Research Center, Agricultural University of Hebei, Baoding, China, and the Ph.D. degree in computer science from RMIT University, Melbourne, VIC, Australia, in 1989, 1992, and 2000, respectively. He is currently a Professor of Computer Science, the Head of the Evolutionary Computation Research Group, and the Associate Dean (Research and Innovation) with the Faculty of Engineering, Victoria University of Wellington, Wellington, New Zealand. His current research interests include evolutionary computation, particularly genetic programming, particle swarm optimization, and learning classifier systems with application areas of image analysis, multiobjective optimization, feature selection and reduction, job-shop scheduling, and transfer learning. He has published over 500 research papers in refereed international journals and conferences. He is a Committee Member of the IEEE NZ Central Section. He is a Fellow of the Royal Society of New Zealand and an IEEE Distinguished Lecturer.